SDN Orchestration of Multi-Domain QKD Networks

The ETSI GS QKD 018 Interface

Diego Carro Bucci Felice Elia

Cybersecurity course, Politecnico di Torino
Quantum Engineering

November 2026

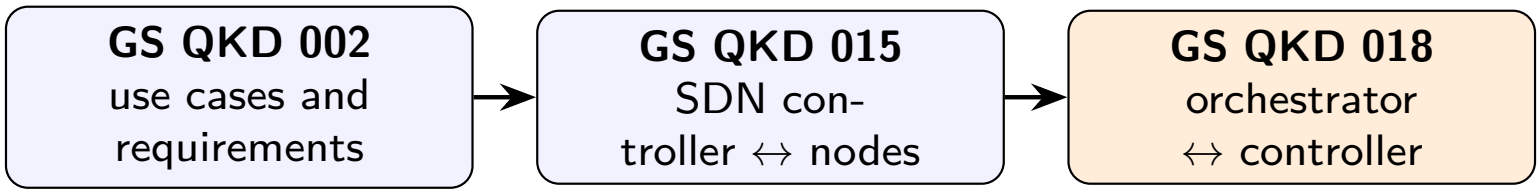
Outline

1. Introduction: from QKD to the network
2. ETSI GS QKD 015: the SDN controller
3. ETSI GS QKD 018: the orchestration interface
4. Real deployments, limits and conclusions

1. Introduction: from QKD to the network
   - Motivation
   - QKD and trusted nodes
   - Architecture and SDN
   - GS QKD 002
2. ETSI GS QKD 015: the SDN controller
3. ETSI GS QKD 018: the orchestration interface
4. Real deployments, limits and conclusions

The problem

- QKD lets two parties agree on a secret key whose security rests on physics, not on computational hardness.
- Bringing it into an operator network means managing it like any other network resource:
  - discover its topology and monitor its state;
  - allocate its capacity to services;
  - coordinate it with the classical optical network.
- The ETSI ISG QKD tackled the problem step by step:



State of the art in brief

Foundations and networks

- Review of protocols and security proofs [1]
- From link to network: layers, trusted relay, routing [2–4]

Standards

- ETSI: use cases [5], APIs [6, 7], SDN [8], orchestration [9], KMS [10]
- ITU-T Y.3800: architecture, control, SDN [11–13]

SDN and orchestration

- First SDN + QKD experiments (UPM/Telefónica) [14, 15]
- KOREN multi-vendor (SK Telecom) [16]
- Hierarchical ONOS prototype (PoliMi) [17]
- MadQCI, Madrid [18]

Contribution

A unified reading of the 002 $\rightarrow$ 015 $\rightarrow$ 018 chain: from requirements to YANG models to multi-domain orchestration.

Quantum key distribution

BB84 in four steps

1. Alice encodes random bits on single photons, in one of two randomly chosen bases.
2. Bob measures in a random basis.
3. Over an authenticated classical channel they compare bases (*sifting*).
4. Error correction and *privacy amplification* distil the key.

An eavesdropper introduces measurable errors:

$$\mathrm{QBER} = \frac{N_{\text{errors}}}{N_{\text{sifted}}} \lesssim 11\% \text{ (BB84)}$$

What the network cares about

The *secret key rate* (SKR), in bit/s, drops with distance:

$$\eta = 10^{-\alpha L/10}$$

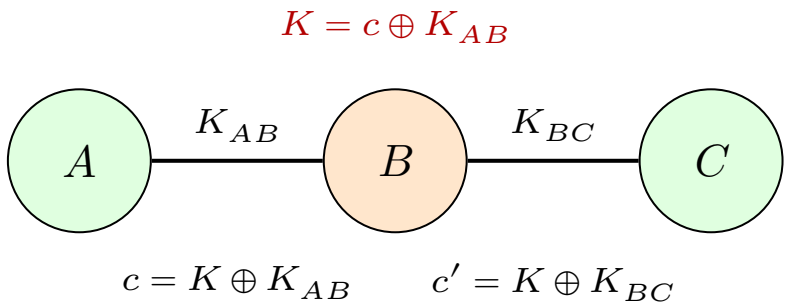
with $\alpha \approx 0.2$ dB/km.

Two remarks

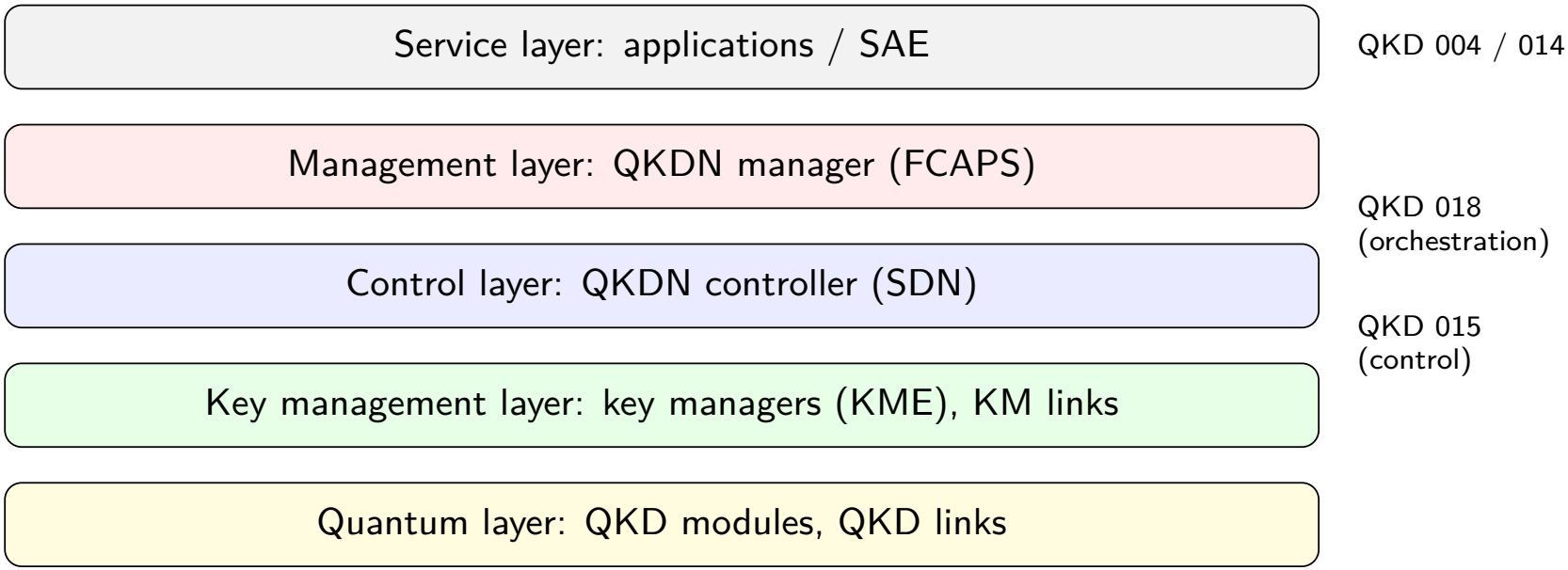
- QKD *distributes* keys; it neither encrypts nor authenticates.
- It needs an initial secret: it is *key growing*.

From link to network: the trusted node

- A link spans at most a few hundred km; practical quantum repeaters do not exist yet.
- Real networks chain links through trusted nodes.
- Each hop is a one-time pad, but K is in the clear inside every intermediate node.
- Relaying consumes key: each link $\ell \in p$ spends $|K|$ bits. Key becomes a resource to be planned.



The ITU-T Y.3800 layered architecture



QKD link: two QKD modules, where key is *generated*.
KM link: two key managers, where key is *relayed*.

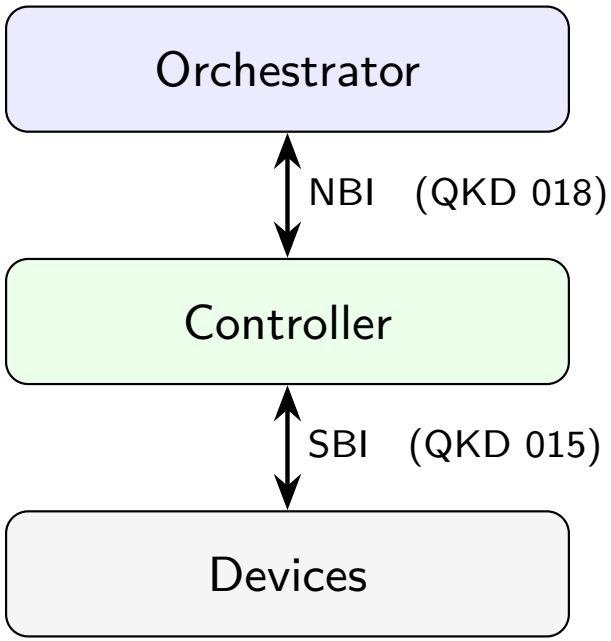
QKDN controller: acts in real time (paths, links).
QKDN manager: supervises (FCAPS, policies).

Adapted from ITU-T Y.3800 [11]

Software-Defined Networking

- SDN separates the *data plane*, the *control plane* (a controller with a global view) and the *application plane* [19].
- The controller talks to devices through the SBI and offers services upwards through the NBI.
- With several domains, an orchestrator sits on top.

A QKD network has everything that makes SDN useful: scarce and variable resources, physical constraints, multi-vendor equipment, changing demand.



GS QKD 002 (2010): where QKD is needed

One primitive out of three

A secure system needs *encryption*, *key distribution* and *authentication*. QKD covers only the second: it complements classical cryptography (OTP/AES, Wegman–Carter MACs) rather than replacing it.

Where the key is consumed

- L2 PPP, MACsec, QKD link encryptor
- L3 modified IKE for IPsec
- L4 TLS session keys, HMAC
- L7 application-level authentication and encryption

In every case there is a key *consumer* and a key *provider*: the future SAE / KME split.

[5]

The six use cases

Case	Configuration	Need it anticipates
UC1 Backup	point-to-point link + line encryptor	key–encryptor interface
UC2 Enterprise metro	several links, key server	per-application QoS
UC3 Critical infrastructure	network with key relay and integrated management	KMS, redundancy, centralised control
UC4 Backbone	quantum channel in WDM	coexistence with the optical network, integrated management
UC5 PON access	point-to-multipoint	sharing of key capacity
UC6 Long-haul	relay through a mobile node (satellite)	trusted node, redundancy

What it leaves behind

Useful QKD is a *network*: key management (UC3, UC6) and differentiated QoS (UC2) ⇒ **GS QKD 015**; integration with the operator's optical network (UC4) ⇒ **GS QKD 018**.

1 Introduction: from QKD to the network

2 ETSI GS QKD 015: the SDN controller
- Components
- YANG model
- Workflows and security

3 ETSI GS QKD 018: the orchestration interface

4 Real deployments, limits and conclusions

Why software-defined control

Today: manual integration

- each system installed by hand on dedicated dark fibre;
- it does not scale;
- key capacity cannot be moved where it is needed;
- no dialogue with the optical network.

Goals of the abstraction

- central management of QKD resources;
- a single key management per perimeter;
- low-level parameters delegated to the node;
- links distributed according to QoS;
- coordination of quantum and classical channels.

GS QKD 015 V2.1.1 (2022) [8]: the southbound interface of the QKD control plane, aligned with 018.

Components of an SD-QKD network

QKD module hardware + software, vendor specific

QKD interface `qkdi`: abstract view of a module (technology, model, λ, optical attachment)

SD-QKD node modules in the same secure location, one *SDN agent*, one key manager, one key access point

Link `qkd_link`: `PHYS` (direct quantum channel) or `VIRT` (key relay)

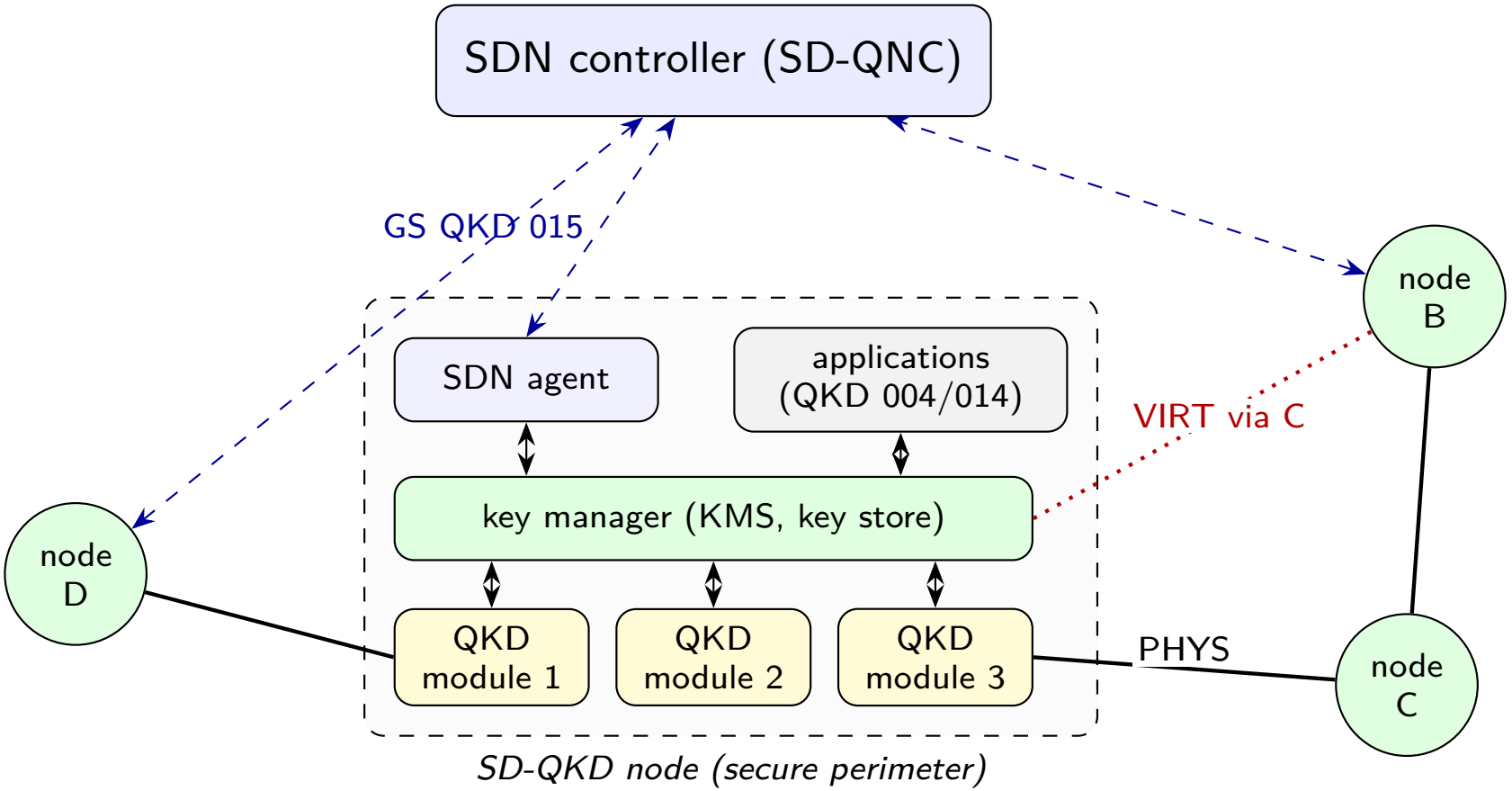
Application `qkd_app`: `CLIENT` or `INTERNAL`, with QoS

Controller SD-QNC: application registry, plans links, talks to the optical network (SD-ONC)

Security principle

Key material *never leaves the node perimeter*: only identifiers, states, rates and physical parameters reach the controller.

The SD-QKD node



Adapted from GS QKD 015, Fig. 1 [8]

Virtual links and applications

`VIRT` key association link

- consumes key from the underlying physical links, so it is an *internal application* with `virt_bandwidth`;
- each intermediate node knows `virt_prev_hop` and `virt_next_hop`;
- requires `key_relay_mode_enable`.

Application QoS

- `max_bandwidth`, `min_bandwidth` (bit/s)
- maximum `jitter` (ms)
- `ttl` of a key in the key store
- sharing of paths or keys

The answer to the differentiated QoS need of UC2.

Anatomy of a YANG model

YANG [20] is a schema (like XSD or JSON Schema) for configuration and state data and for operations.

- `leaf`, `leaf-list`, `container`, `list` with `key`
- `config true` (the controller decides) / `config false` (the node measures)
- `grouping`/`uses` to reuse, `augment` to extend another module
- `leafref`, `when`, `must` for constraints
- `rpc` and `notification`

YANG: module header and identities

```
module etsi-qkd-sdn-node {
  namespace "urn:etsi:qkd:yang:etsi-qkd-node";
  prefix "etsi-qkdn";
  import etsi-qkd-node-types { prefix "etsi-qkdn-types"; }
  revision "2022-01-30" { reference "ETSI␣GS␣QKD␣015␣V2.1.1"; }
  ...
}
identity QKD-LINK-TYPES { description "QKD␣key␣association␣link␣types."; }
identity VIRT { base QKD-LINK-TYPES; }
identity PHYS { base QKD-LINK-TYPES; }
typedef qkd-link-types {
  type identityref { base QKD-LINK-TYPES; }
}
```

Unlike an `enumeration`, `identity` values can be extended by other modules.

The node model: etsi-qkd-sdn-node

```
module: etsi-qkd-sdn-node
  +--rw qkd_node
     +--rw qkdn_id                yang:uuid
     +--ro qkdn_status?           etsi-qkdn-types:qkdn-status-types
     +--rw qkdn_capabilities
     |  +--rw link_stats_support?  application_stats_support?
     |  +--rw key_relay_mode_enable?
     +--rw qkd_applications
     |  +--rw qkd_app* [app_id]
     |     +--rw app_id, app_type, server_app_id, client_app_id*
     |     +--rw app_qos             (max/min_bandwidth, jitter, ttl)
     |     +--rw backing_qkdl_id*, local_qkdn_id, remote_qkdn_id
     +--rw qkd_interfaces
     |  +--rw qkd_interface* [qkdi_id]
     |     +--rw qkdi_model, qkdi_type, qkdi_att_point
     |     +--rw qkdi_capabilities  (role, wavelength_range, ...)
     +--rw qkd_links
        +--rw qkd_link* [qkdl_id]
           +--rw qkdl_type, qkdl_enable, qkdl_local / qkdl_remote
           +--rw virt_prev_hop, virt_next_hop*, virt_bandwidth (VIRT)
           +--rw phys_channel_att, phys_wavelength          (PHYS)
           +--rw qkdl_performance (skr, eskr, expected_consumption)
```

The node model: sections and performance

One tree per node, four sections:

- identity and **capabilities**;
- **applications** with QoS;
- **interfaces** (modules);
- **links** with performance.

Performance of link ℓ

$$\mathrm{EC}_\ell = \sum_{a \in \mathcal{A}_\ell} b_a$$

$$\mathrm{ESKR}_\ell \approx \mathrm{SKR}_\ell - \mathrm{EC}_\ell$$

EC = `expected_consumption`; an intuitive reading, not a formula of the standard.

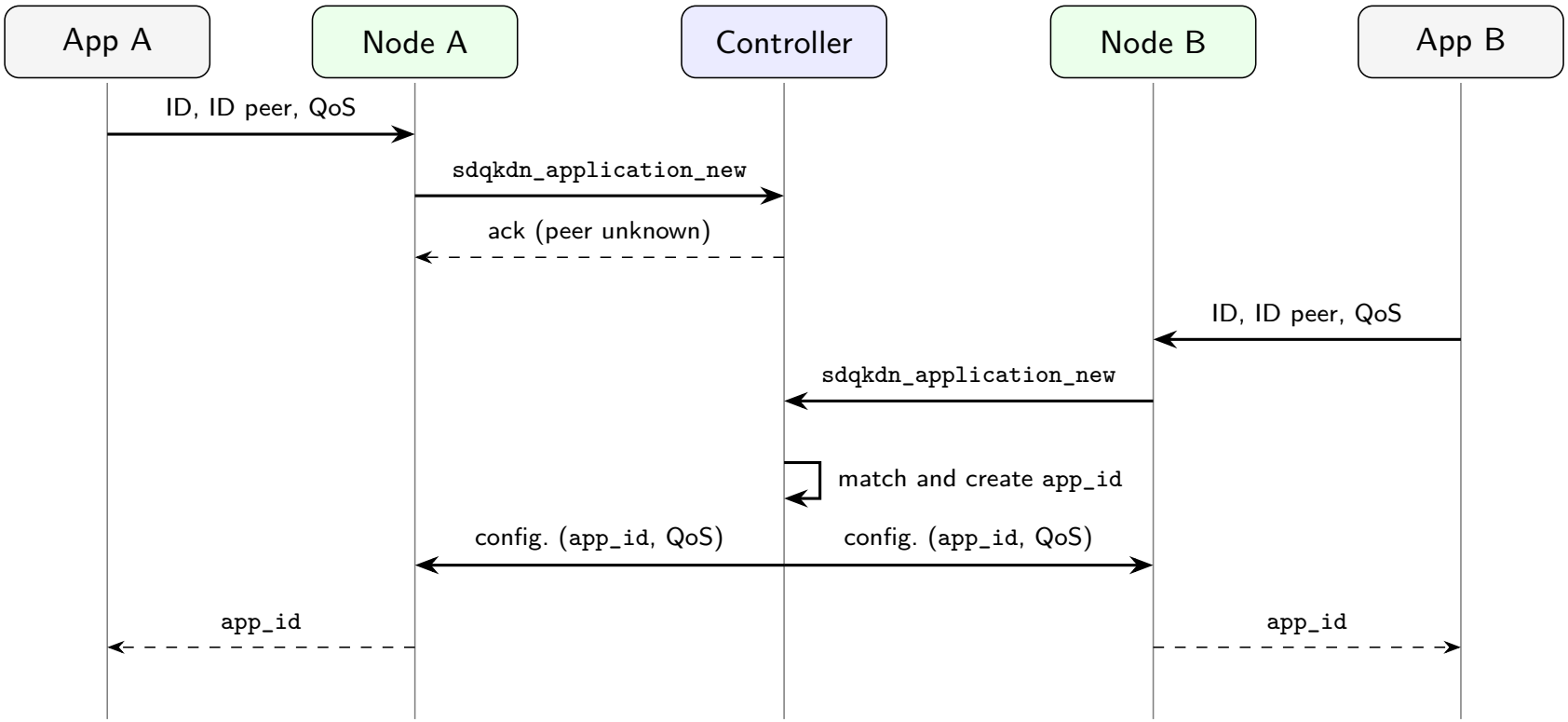
Notifications from node to controller

Polling nodes continuously is inefficient: 015 defines YANG notifications per object.

Object	**Notifications**
Applications	`sdqkdn_application_new`, `_qos_update`, `_disconnected`
Interfaces	`sdqkdn_interface_new`, `_down` (with *reason*), `_out`
Links	`sdqkdn_link_down`, `_perf_update`, `_overloaded`
Generic	extensible events and alarms

NETCONF/RESTCONF also allow subscribing to any datastore node and receiving every change in real time.

Workflow: application registration



Applications only know their own key manager: the controller, acting as central registry, matches the requests and creates the app_id.

Adapted from GS QKD 015, Fig. 2 [8]

Workflow: creating physical and virtual links

Physical link

1. trigger (demand at the nodes or NBI);
2. two free, compatible interfaces;
3. *lightpath* request to the SD-ONC;
4. interface configuration.

The core of coexistence with the optical network (UC4).

Virtual link

Without a lightpath: a path over the QKD network, intermediate nodes in relay mode, endpoints with the requested bandwidth. Example (ours, not part of the standard):

$$w_\ell = \frac{1}{\mathrm{ESKR}_\ell}$$

$$p^\star = \arg\min_{p \in \mathcal{P}_r} \sum_{\ell \in p} w_\ell$$

$$\mathcal{P}_r = \left\{ p : \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r \right\}$$

Badly managed virtual links congest the physical ones: the controller's global view is needed.

Protocols, security and limits of 015

Protocols

- NETCONF [21]: device–controller, transactional, SSH/TLS, XML
- RESTCONF [22]: NBI, HTTP, CRUD, JSON/XML

Secure installation

- existing secure area: channel protected with QKD keys;
- new site: initial secret, then QKD keys (*key growing*).

What is left out

015 controls *one* domain with *one* controller.

- several QKD domains, possibly from different vendors;
- SAEs in the nodes of another network (OTN);
- management with the same orchestration tools as the rest of the network.

$\Rightarrow$ **GS QKD 018**

1 Introduction: from QKD to the network

2 ETSI GS QKD 015: the SDN controller

3 ETSI GS QKD 018: the orchestration interface
- Scenario and model
- YANG modules
- Workflow and security
- ACTN analogy

4 Real deployments, limits and conclusions

GS QKD 018 V1.1.1 (2022)

- Interface between the SDN orchestrator (client) and the SDN controller of a QKD domain (server) [9].
- It is the controller's *northbound interface*: the complement of 015.

$$\text{orchestrator} \xrightarrow{\text{QKD 018}} \text{controller} \xrightarrow{\text{QKD 015}} \text{SD-QKD node} \xrightarrow{\text{QKD 004/014}} \text{SAE}$$

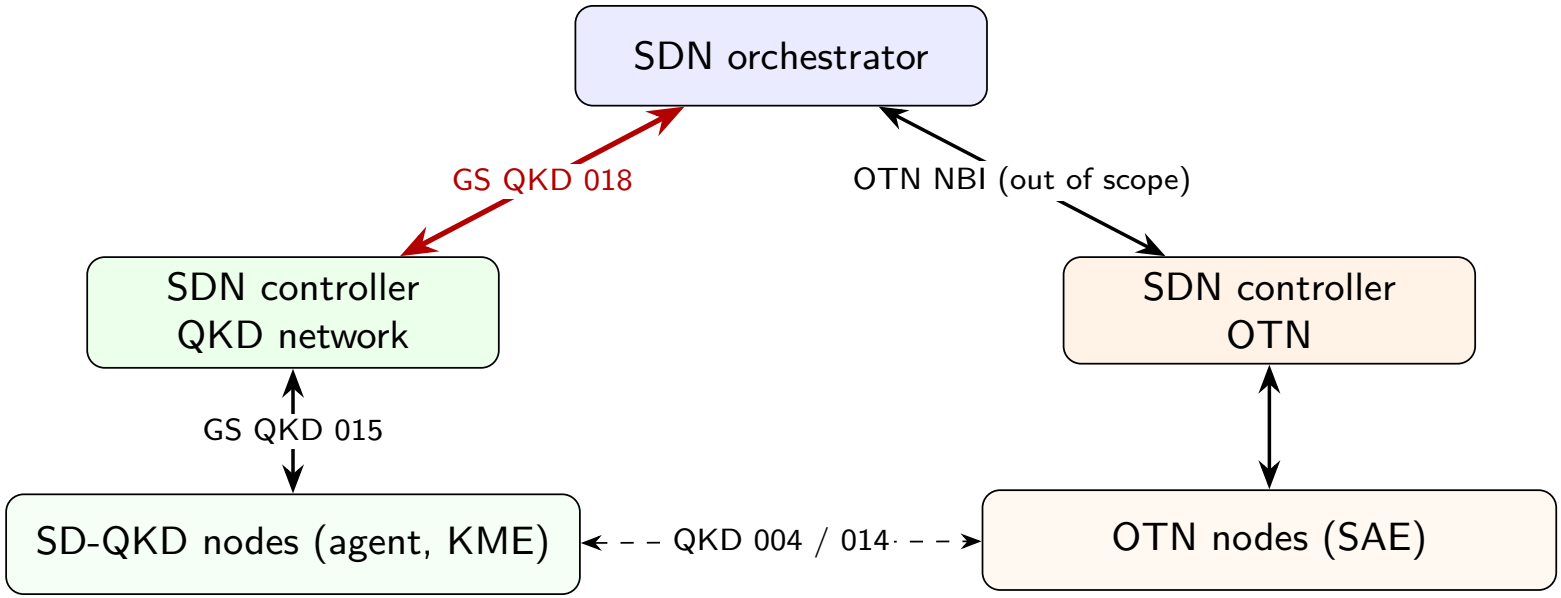
SDN orchestration

Continuous, automatic coordination of resources to set up and release end-to-end services across several domains, each with its own controller.

Service link

Logical key association between two QKD nodes, single- or multi-hop: at orchestration level it is the 015 virtual link.

The reference scenario



- QKD network separate from the optical transport network (OTN): performance, isolation, separate responsibilities.
- SAEs sit in OTN nodes, keys in QKD nodes; each controller only sees its own network.

The orchestrator solves:

1. *address matching* OTN node $\leftrightarrow$ QKD node;
2. end-to-end provisioning;
3. management: topology, inventory, FCAPS.

Adapted from GS QKD 018, Fig. 1 [9]

Orchestrator functions and information model

Required functions (§4.2)

- translate requests into domain configuration;
- activate and release inter-domain services;
- compute and modify multi-domain paths;
- discover nodes and links, abstract view;
- monitor key resources and FCAPS.

Five information blocks (§5)

§5.1 topology discovery, proactive or reactive

§5.2 state and inventory: SKR, ESKR, restoration

§5.3 state of end-to-end services

§5.4 provisioning with candidate path computation

§5.5 notifications and configurable thresholds (QBER, SKR, ESKR)

Path admissibility (our formalisation)

Every link of a multi-hop path relays the same key, so the bottleneck is what matters:

$$p \text{ admissible for request } r \iff \min_{\ell \in p} \mathrm{ESKR}_\ell \geq r$$

Three YANG modules: topology

```
qkd_network
 +-- sdqkd_nodes
 |    +-- qkdn* [qkdn_id]                (SD-QKD nodes
      )
 |         +-- qkd_interfaces
 |              +-- qkdi* [qkdi_id]      (QKD modules)
 +-- qkd_links
      +-- qkd_phys_links
      |    +-- phys_link* [phys_link_id]
      |         +-- link_type
      |         +-- phys_link_local_qkdn
      |         |    (qkdn_id_ref, qkdi_id_ref)
      |         +-- phys_link_remote_qkdn
      +-- qkd_svc_links
           +-- svc_link* [svc_link_id]
                +-- ... local/remote endpoints
```

Chain: *topology* → *inventory* → *connectivity*. The modules import the 015 types.

`etsi-qkd-sdn-topology`: a single tree for the whole domain, i.e. a graph

$$G_i = (V_i, E_i), \quad E_i = E_{\text{phys}} \cup E_{\text{svc}}$$

with validated `leafref` endpoints. The orchestrator builds the multi-domain view:

$$G = \left(\bigcup_i V_i, \ \bigcup_i E_i \cup E_{\text{inter}} \right)$$

E_{inter}: adjacencies between domains, typically border nodes.

[23]

Three YANG modules: inventory

`etsi-qkd-sdn-inventory` does not create a new tree: it `augment`s the topology.

Element	Added leaves
`qkdn`	version, capabilities, location, `KME_ip_port` (where SAEs fetch keys via QKD 014), state
`qkdi`	`role_support`, model, type, attachment point, state
`phys_link`	`enable`, `wavelength`, `performance` (`skr`, `eskr`, `expected_consumption`, `phys_link_perf`: QBER, SNR), state
`svc_link`	`bandwidth`, `performance`, `path_restoration` (`AUTOMATIC`/`MANUAL`, `hold_off_time`), state

Notifications: `sdqkdn_new`, `_interface_new`, `_phys_link_new`, `_svc_link_new`, the `_perf_update` and `_overloaded` variants for physical and service links, `alarm` and `event` (node status travels here).

Three YANG modules: connectivity

```
connectivity_service/qkd_service
 +-- qkd_app* [app_id]
      +-- (app data from QKD 015: state, QoS)
      +-- app_mapped_svc_link
           +-- svc_link_and_qkdn_ids
           +-- path_constraints
           |    (include/exclude_nodes:
           |     node, interface pairs)
           +-- candidate_paths
           |    +-- candidate_path* [path_id]
           +-- deploy_path (path_id, phys_links)
```

Division of roles

The controller knows its domain and proposes paths; the orchestrator has the overall view and decides.

RPCs: the provisioning workflow step by step

1 app-registration
2 create-service-link-id-for-app
3 create-candidate-paths-per-service-link-for-app
3' update-candidate-paths-per-service-link-for-app
4 deploy-a-candidate-path-per-service-link-for-app
5 delete-deployed-path-per-service-link-for-app

Notifications

sdqkdn_application_new, _deployed, _qos_update, _disconnected

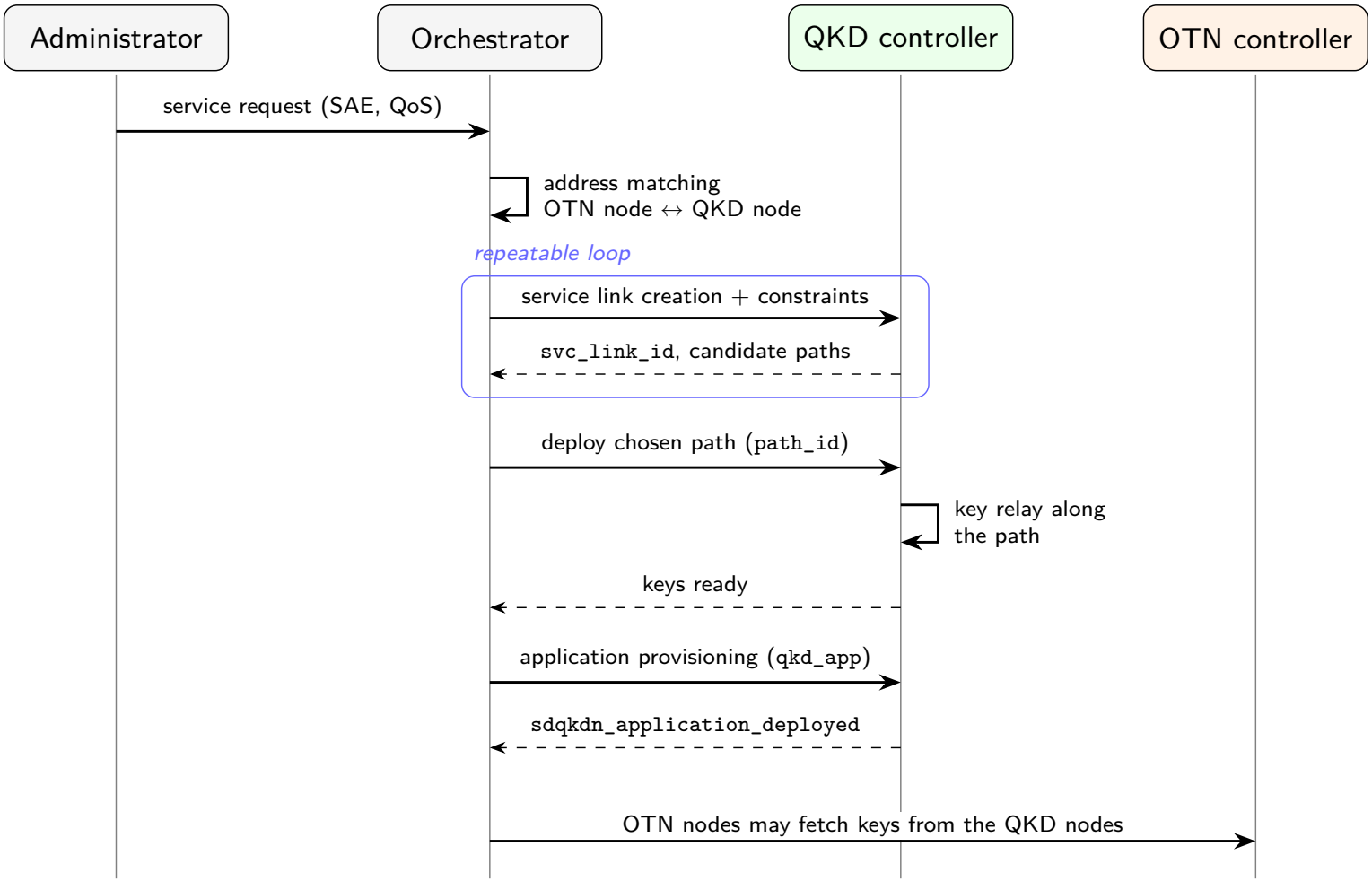
The three modules at a glance

Module	Question	Clause	Data
topology	What does the network look like?	§5.1	structure (graph)
inventory	How many resources, in what state?	§5.2, §5.5	state, performance, configuration, notifications
connectivity	How do I serve an application end to end?	§5.3, §5.4, §6.2	applications, path computation and deployment RPCs

Why this structure

Whoever only needs the structure implements *topology*; whoever also wants state adds *inventory* via `augment`, with no duplication; *connectivity* adds the operations.

QKD + OTN provisioning workflow (§6.2)



Simplified rework of GS QKD 018, Fig. 2 [9]

The workflow step by step

1. The administrator requests a service (SAEs, QoS).
2. The orchestrator identifies the controllers and performs address matching.
3. It asks the QKD controller for a service link and paths, with constraints.
4. The controller assigns a `svc_link_id` and returns prioritised candidates (repeatable step).
5. The orchestrator picks one and requests its deployment; the nodes relay keys along the path.
6. Application provisioning and notice to the OTN controller.

015 vs 018

In 015 the controller computes and configures the virtual link by itself; in 018 it proposes and waits for the choice.

Several QKD domains

The orchestrator repeats steps 3–5 for each domain along the inter-domain path, joining segments at the border nodes.

Protocols and security

Transport (§8)

- Model (YANG 1.1) separate from the protocol: NETCONF or RESTCONF.
- RESTCONF: every resource is a URI; `GET` to read, `POST`/`PUT`/`PATCH`/`DELETE` to configure; notifications via streams.

Security (§7)

QKD keys **never transit** the controller or the orchestrator: the models only carry rates and the KME address. A compromised orchestrator can cause outages, not read keys. Channel protection (e.g. TLS) is left to the implementation.

```
GET  /restconf/data/etsi-qkd-sdn-topology:qkd_network
POST /restconf/operations/etsi-qkd-sdn-connectivity:create-service-link-id-for-app
```

018 applies the ACTN pattern to QKD

ACTN	ETSI QKD	Remark
CNC	administrator / customer portal	originates the request
MDSC	SDN orchestrator	multi-domain view, path decision
PNC	SDN controller of the QKD domain	computes candidates, activates paths
MPI	GS QKD 018 interface	abstract topology, provisioning, notifications
SBI	GS QKD 015 interface	control of individual nodes
abstract TE topology	`etsi-qkd-sdn-topology`	vendor-neutral view
TE bandwidth	SKR / ESKR	the resource to allocate is key

The difference: the resource

In ACTN bandwidth is fixed by channel capacity; in QKD the SKR *fluctuates* and key accumulates in finite buffers. Hence the weight of `_perf_update`, `_overloaded` and `path_restoration`.

[24]

1 Introduction: from QKD to the network

2 ETSI GS QKD 015: the SDN controller

3 ETSI GS QKD 018: the orchestration interface

4 Real deployments, limits and conclusions
- SK Telecom
- PoliMi and MadQCI
- Limits
- Conclusions

SK Telecom: multi-vendor orchestration on KOREN

- First documented deployment of a 018-compliant orchestrator (2020–2022) [16].
- Two operators, KMS and devices from different vendors, each with its own controller (QKDNC); one orchestrator (QKDNO).
- Decoy-state BB84, spans from 39 to 100 km.
- Border nodes (Pangyo, Daejeon): no QKD modules, two interconnected KMSs.

Model extensions

- *multi-segment* application type (`MS`), with an `ms_app_id` shared across domains;
- `server_ms_info`, `client_ms_info`;
- `app-registration-ms` RPC;
- keys at the border via GS QKD 020 [10].

`CLIENT`/`INTERNAL` are not enough for multi-domain; YANG `identity` makes extension possible.

PoliMi and MadQCI

Politecnico di Milano [17]

- Hierarchical control for metropolitan networks.
- ONOS-based domain controllers with the 015 model.
- 018 interface over RESTCONF towards a Python orchestrator aggregating several domains.
- The whole 015 $\rightarrow$ 018 stack in open software.

MadQCI, Madrid [18]

- Real network: dozens of multi-vendor modules over more than 130 km, integrated with the production optical network.
- SD-QKD control based on 015.
- 018 described as *ongoing* work.

Domain control is mature; orchestration much less so.

Limits and outlook

Adoption 015 runs in real networks, 018 almost only in testbeds.

Multi-domain the normative scenario is QKD + OTN; QKD + QKD needs extensions and GS QKD 020.

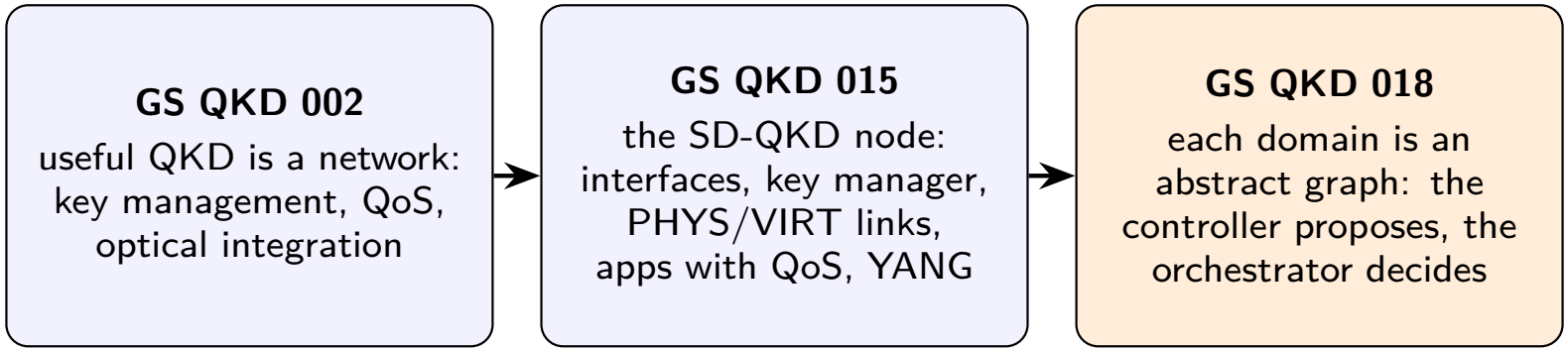
Metrics ESKR is instantaneous and ignores buffers [25].

Confidentiality exposing topology is fine within one operator, less so across operators [25, 26].

Security protection of the orchestrator–controller channel is not specified.

Algorithms path computation is not standardised [4, 27].

Conclusions



- The design mirrors ACTN and reuses YANG, NETCONF and RESTCONF.
- Key principle: keys never leave the nodes.
- KOREN, PoliMi and MadQCI: the scheme works, but orchestration is young. Next step: multi-operator, better metrics, confidentiality across domains.

References I

[1] Stefano Pirandola et al. "Advances in quantum cryptography". In: *Advances in Optics and Photonics* 12.4 (2020).

[2] Miralem Mehic et al. "Quantum Key Distribution: A Networking Perspective". In: *ACM Computing Surveys* 53.5 (2020).

[3] Yuan Cao et al. "The Evolution of Quantum Key Distribution Networks: On the Road to the Qinternet". In: *IEEE Communications Surveys & Tutorials* (2022).

[4] *QKD Routing Protocol in Quantum Networks: Overview and Challenges*. arXiv:2407.13156. 2024.

[5] ETSI ISG QKD. *Quantum Key Distribution (QKD); Use Cases*. Group Specification GS QKD 002 V1.1.1. ETSI, June 2010.

[6] ETSI ISG QKD. *Quantum Key Distribution (QKD); Application Interface*. Group Specification GS QKD 004 V2.1.1. ETSI, 2020.

[7] ETSI ISG QKD. *Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API*. Group Specification GS QKD 014 V1.1.1. ETSI, 2019.

[8] ETSI ISG QKD. *Quantum Key Distribution (QKD); Control Interface for Software Defined Networks*. Group Specification GS QKD 015 V2.1.1. ETSI, Apr. 2022.

[9] ETSI ISG QKD. *Quantum Key Distribution (QKD); Orchestration Interface for Software Defined Networks*. Group Specification GS QKD 018 V1.1.1. ETSI, Apr. 2022.

[10] ETSI ISG QKD. *Quantum Key Distribution (QKD); Interoperable Key Management System*. Group Specification GS QKD 020. ETSI.

References II

[11] ITU-T. *Overview on networks supporting quantum key distribution*. Recommendation Y.3800. International Telecommunication Union, 2019.

[12] ITU-T. *Quantum key distribution networks – Control and management*. Recommendation Y.3804. International Telecommunication Union, 2025.

[13] ITU-T. *Quantum key distribution networks – Software-defined networking control*. Recommendation Y.3805. International Telecommunication Union, 2021.

[14] Alejandro Aguado et al. "First experimental demonstration of secure NFV orchestration over an SDN-controlled optical network with time-shared QKD resources". In: *European Conference on Optical Communication (ECOC)*. arXiv:1604.05861. 2016.

[15] Alejandro Aguado et al. "Enabling Quantum Key Distribution Networks via Software-Defined Networking". In: *International Conference on Optical Network Design and Modeling (ONDM)*. 2020.

[16] Dong-Hi Sim, Jongyoon Shin, and Min Hyung Kim. "Software-Defined Networking Orchestration for Interoperable Key Management of Quantum Key Distribution Networks". In: *Entropy* 25.6 (2023), p. 943.

[17] E. Vicario. "Hierarchical SDN control for advanced quantum-secured metropolitan networks". MA thesis. Politecnico di Milano, 2025.

[18] A. Sebastián-Lombraña et al. *A blueprint for large-scale quantum-network deployments*. arXiv:2409.01069. 2024.

[19] Diego Kreutz et al. "Software-Defined Networking: A Comprehensive Survey". In: *Proceedings of the IEEE* 103.1 (2015).

References III

[20] Martin Bjorklund. *The YANG 1.1 Data Modeling Language*. RFC 7950. IETF, 2016.

[21] Rob Enns et al. *Network Configuration Protocol (NETCONF)*. RFC 6241. IETF, 2011.

[22] Andy Bierman, Martin Bjorklund, and Kent Watsen. *RESTCONF Protocol*. RFC 8040. IETF, 2017.

[23] ETSI ISG QKD. *gs018-orch-int-sdn: YANG modules etsi-qkd-sdn-topology, etsi-qkd-sdn-inventory, etsi-qkd-sdn-connectivity (rev. 2022-02-22)*. ETSI Forge. 2022.

[24] Daniele Ceccarelli and Young Lee. *Framework for Abstraction and Control of TE Networks (ACTN)*. RFC 8453. IETF, 2018.

[25] S. Laschet et al. "SDN Key Relay for Large-Scale QKD Networks". In: *IEEE OCNC*. arXiv:2608.17539. 2026.

[26] *Interconnecting Regional QKD Networks: Hybrid Key Delivery Across Quantum Domains*. arXiv:2604.20376. 2026.

[27] R. B. Méndez et al. "Switching Coordinator: An SDN Application for Flexible QKD Networks". In: *Entropy* 28 (2026). arXiv:2603.13812, p. 219.

Thank you for your attention

Questions?